COL215 Lab Assignment 4b

Hamming-Distance and Similarity Engine

ANSH : 2025CS51207 & Priyam Gupta : 2025CS11135

1. Design Decisions

In this lab, we extended our Lab 4a Hamming-distance circuit and made a wrapper so that we can use it on the Basys3 FPGA board. We use the switches to enter the two 8-bit values and store them in two registers. Then we give these values to the hamming_distance8 module. We use LEDs to show the bit difference and the seven-segment display to show the Hamming distance.

Module 1: mux2_1_8bit

Inputs: in0[7:0], in1[7:0], sel

Output: out[7:0]

We use this module to select one of two 8-bit inputs. When sel is 0, the output is in0. When sel is 1, the output is in1. We use two such muxes in the wrapper to generate next_A and next_B.

Module 2: hamming_wrapper

Inputs: clk, sw[15:0]

Outputs: led[15:0], an[3:0], seg[6:0], dp

We use SW[7:0] as the common 8-bit data input. SW[8] decides whether we are storing A or B. SW[12:9] is used as the 4-bit threshold. SW[15:13] are not used in our design.

Capturing A and B

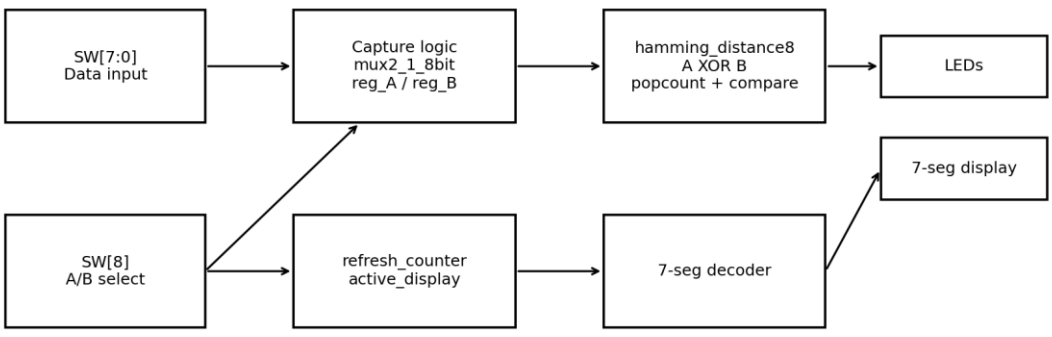
We use two 8-bit registers, reg_A and reg_B, to store the two operands. When SW[8] = 0, next_A gets SW[7:0] and reg_B keeps its old value. When SW[8] = 1, next_B gets SW[7:0] and reg_A keeps its old value. The registers update at the positive edge of the clock. Therefore, this part of our design is sequential.

Hamming-distance calculation

We connect reg_A and reg_B to the hamming_distance8 module from Lab 4a. First, we calculate difference = A ^ B. The 1s in the difference show the bit positions where A and B are different. We then use popcount8 to count these 1s and get the Hamming distance.

We also use comparator8_eq to check whether the difference is zero. This gives equal = 1 when A and B are the same. We compare the distance with SW[12:9] using comparator4. We get within_threshold = 1 when distance <= threshold.

Figure 1. Simple block diagram of our hamming_wrapper



2. Combinational and Sequential Parts

We have both combinational and sequential logic in our wrapper.

Sequential parts

- reg_A and reg_B store the two operands.
- refresh_counter is used to refresh the seven-segment display.
- active_display stores which display digit is currently active.

These parts use the clock. The values change only at clock edges.

Combinational parts

- The two mux2_1_8bit modules select the next values of A and B.
- hamming_distance8 calculates difference, distance, equal and within_threshold.
- LED assignments directly connect the calculated outputs to the LEDs.
- The seven-segment decoder generates the segment pattern for the selected digit.

The Hamming-distance calculation itself does not need a clock. We calculate it using combinational logic from the stored values of A and B.

3. Output Connections

We use the LEDs to see the result directly on the board.

- LED[7:0] shows difference = A ^ B.
- LED[15] shows equal.
- LED[14] shows within_threshold.
- LED[13:8] are kept at 0.

We use the four-digit seven-segment display to show the Hamming distance. The refresh_counter and active_display select the digit one at a time. The display is refreshed continuously so that all four digits appear active to us.

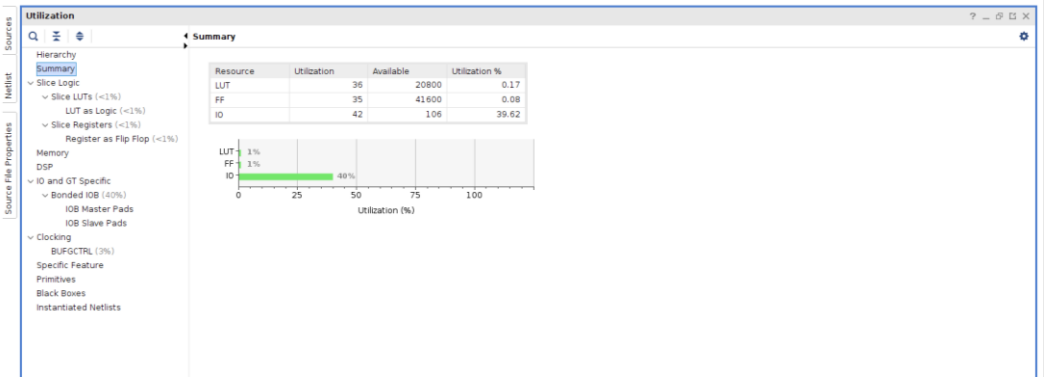
Seven-segment display

- The rightmost digit shows the Hamming distance.
- The next digit shows 0.
- The two left digits show '-'.
- The decimal point is kept off.

Our refresh counter runs from the 100 MHz board clock. After the required number of clock cycles, it changes the active digit. This allows us to refresh all four digits.

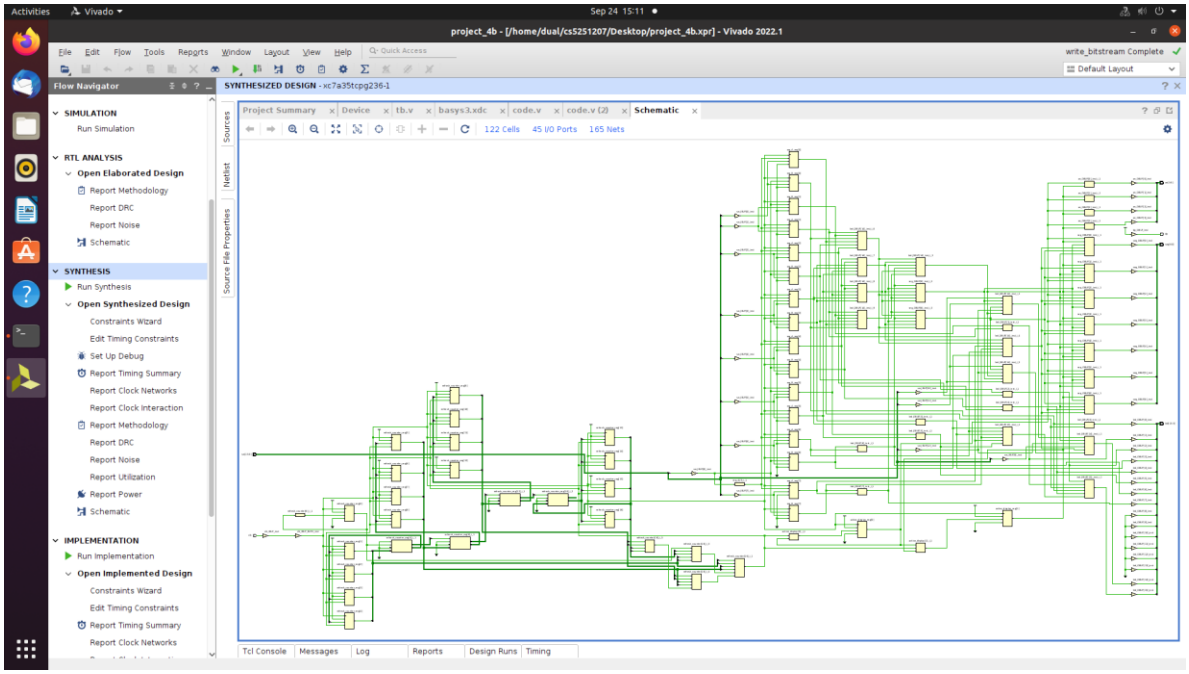
4. FPGA Resource Utilization

We checked the resource utilization after synthesis and implementation in Vivado. We use the values reported by Vivado for the final submitted design.



5. Synthesized Schematic

We use the synthesized schematic to see how our design is connected after synthesis. We can identify the registers, the hamming_distance8 block, the LED path, and the seven-segment display path.



6. FPGA Demonstration

For the board demonstration, we first set SW[7:0] to the value of A and keep SW[8] = 0. After a clock edge, this value is stored in reg_A. Then we change SW[7:0] to the value of B and set SW[8] = 1. After the next clock edge, B is stored in reg_B while A remains unchanged.

After storing both values, we set SW[12:9] according to the required threshold. The lower 8 LEDs show A XOR B. LED[15] shows whether A and B are equal, and LED[14] shows whether the Hamming distance is within the threshold. The seven-segment display shows the distance.

Example

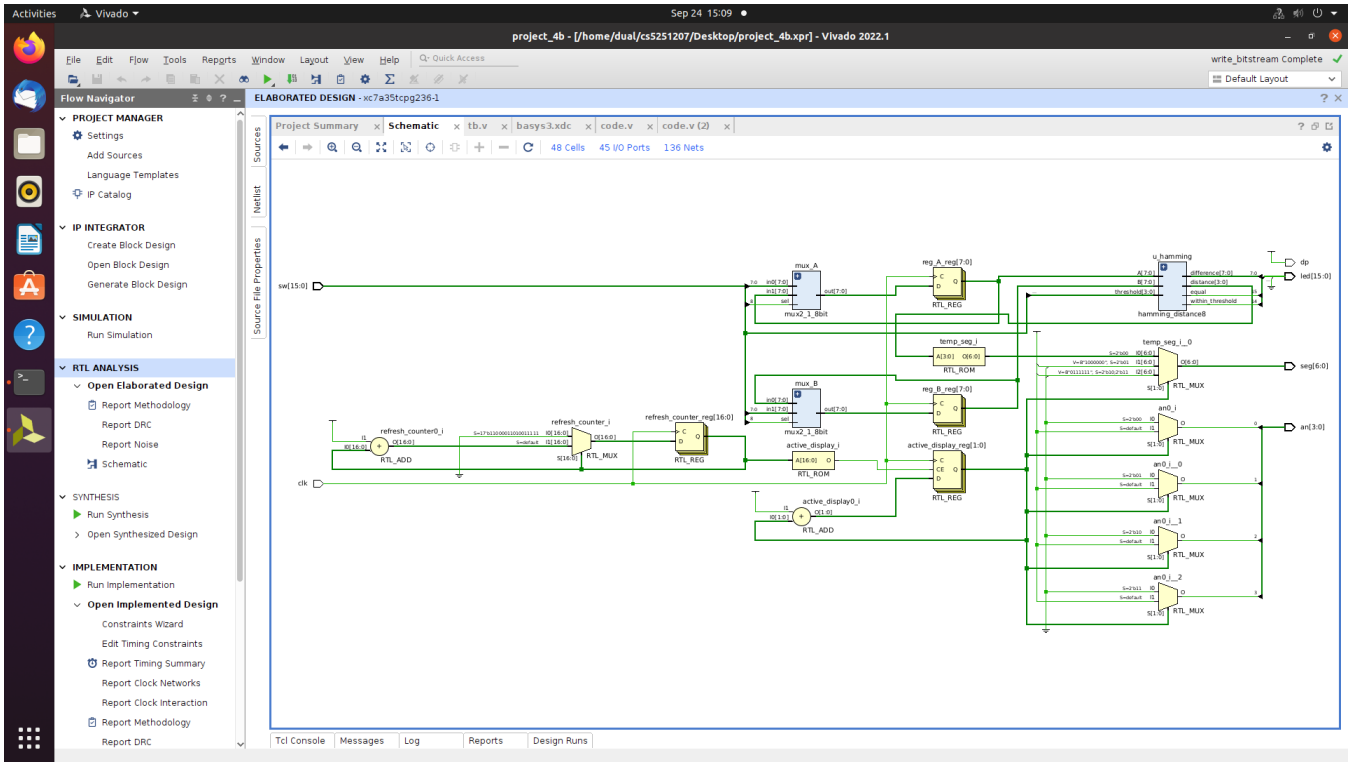
For example, if A = 10110100 and B = 10011100, then:

```
A          = 10110100
B          = 10011100
A XOR B    = 00101000
Hamming distance = 2
```

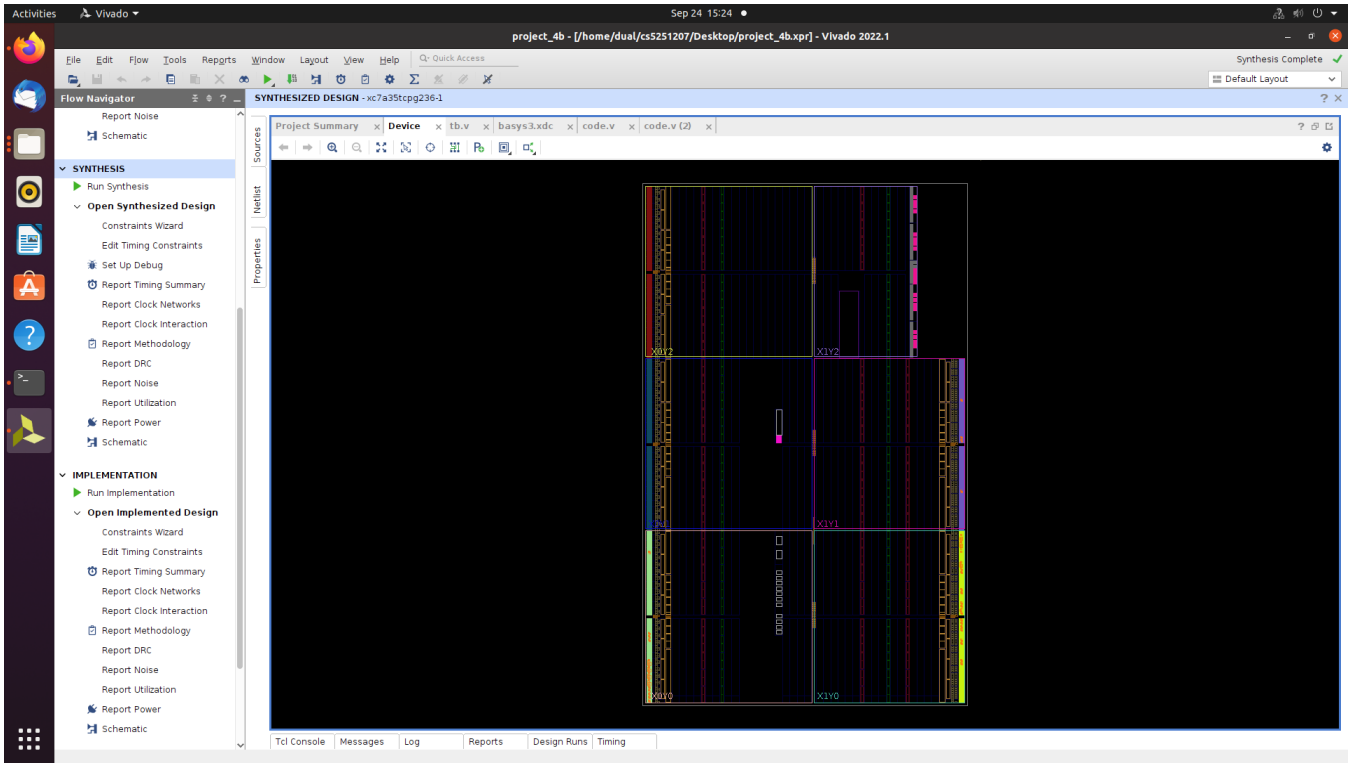
There are two 1s in the difference, so the Hamming distance is 2. If the threshold is 3, then within_threshold becomes 1 because 2 <= 3.

7. Conclusion

In this lab, we used our Lab 4a Hamming-distance circuit with a Basys3 wrapper. We added registers to store A and B, used switches for input and threshold, and added LEDs and a seven-segment display for output. We also checked the synthesized design and FPGA resource usage in Vivado.



Block Diagram



Synthesis Design